UNIVERSITÀ
DEGLI STUDI
FIRENZE
Da un secolo, oltre.

Scuola di
Ingegneria

Corso di Laurea in
Ingegneria Informatica

Development of a semi-automatic annotation system of anomalies in thermal videos

Relatore
Marco Bertini

Candidata
Angela Liguori

Anno Accademico 2024/2025

Contents

1 Theoretical Background **5**

1.1 Computer Vision and Segmentation 5

1.1.1 Sensing Device 5

1.1.2 Interpreting Device 5

1.2 Anomaly Detection 6

1.2.1 Thermal videos 7

1.3 Annotation 8

1.3.1 Image Annotation 8

1.3.2 Video Annotation 8

1.3.3 Costs and Challenges 10

1.4 Human-In-The-Loop Paradigm 11

1.4.1 Active Learning 11

1.4.2 Interactive Machine Learning 13

1.4.3 Machine Teaching 15

2 Development of the Annotation System **17**

2.1 Addressing Annotation Challenges 17

2.2 The HITL Approach in the System 17

2.3 System Architecture 18

2.3.1 Hardware Infrastructure 18

2.3.2 Software Architecture 20

2.4 User Workflow 24

2.4.1 Phase 1: Video Upload and Visualisation 26

2.4.2 Phase 2: Automated Pre-screening 26

2.4.3 Phase 3: Interactive Annotation 26

2.4.4 Phase 4: Automated Annotation 27

2.4.5 Phase 5: The Loop 27

2.4.6 Phase 6: Output Export 27

2.5 Models Integration 27

2.5.1 Integration of RFI Anomaly Detection 28

2.5.2 Integration of YOLOE-26L-Seg 29

2.5.3 Integration of SAM 3 30

3 Models Presentation **33**
3.1 RFI anomaly detection 33
3.2 YOLOE-26L-Seg 33
3.3 SAM 3 34

4 Conclusion **37**
4.1 System Considerations 37
4.2 Models Analysis 38
4.2.1 RFI Anomaly Detection 38
4.2.2 The Segmentation Models 38
4.3 Future Expansions and Improvements 41

References **43**

Abstract

Whether an artificial intelligence model is performative, efficient and accurate or not, is directly linked to the quality of its learning process. A well trained model is expected to generally perform well, and this is possible by reason of the availability of high quality annotated datasets to use as training sets.

A critical feature to consider about data in the training context is the accurate representation of the specialised domains in which the model is meant to be used.

In the computer vision field, training datasets are mostly comprised of optic images and therefore lacking in the thermal domain. This makes classically trained computer vision models weak when facing such media, as they lack standard visual features like colour and fine texture.

Annotating a tailored thermal dataset to use for training is the solution to the lacking problem, which, however, opens the door to other challenges and specific needs.

This thesis presents the development of a system addressing these problems: the laborious nature of manual annotation and the insufficient reliability of automated annotation, specifically in safety-critical domains such as railways infrastructures, the particular environment taken into consideration, to which the specific application of anomaly detection is to be added, along with all its perks.

The application's function is annotating thermal videos for anomaly detection through an interactive loop of mask generation and correction, handled via Web User Interface, whose core functions depends on three different models used at different stages for different purposes: Grounded SAM 2 in the RFI Anomaly Detection algorithm, YOLOE-26L-Seg and SAM 3.

With the use of the Human-In-The-Loop paradigm, the user can directly interact with a computer vision model which dynamically responds by creating and refining masks to identify and track thermal anomalies.

Ultimately, the aim of this project is to lay down the basis for the creation of a tool that leverages human cognitive ease in recognition alongside machine computational speed, making the annotation process, particularly within the complex thermal domain, substantially more efficient.

1 Theoretical Background

This section breaks down the primary topics involved in the development of the application at a theoretical level.

1.1 Computer Vision and Segmentation

Computer vision is the field of AI that uses visual perception to understand the environment and, consequently, take actions based on those perceptions.

An AI vision system is made up of two main components: a sensing device and an algorithm to interpret the image's content.

1.1.1 Sensing Device

In light of the project's domain being thermal videos, the sensing device used for acquiring such videos is a thermal camera mounted on a rotating drone or a moving train.

1.1.2 Interpreting Device

The computer vision algorithm acts as the brain of the vision system, which can be shaped in different ways depending on the context. There are three main approaches that get progressively more accurate in terms of what kind of information they seek to acquire from an image.

Image classification assumes the presence of a single target object in the image to be recognised and classified.

Object detection has the ability to classify and locate multiple targets in one image, enclosing them in labeled rectangles called bounding boxes, which allows the algorithm to pinpoint the targets.

Segmentation refines the object's location information by analysing and classifying every pixel of the image in order to create masks that follow the exact shape of each target. The two branches it divides into are semantic and instance, the latter being able to distinguish between single targets even when they belong to the same class of objects.

1.2 Anomaly Detection

Anomalies are events that deviate from what is normal, usual and thus expected, and they happen without notice. Furthermore, the effects they cause are typically unknown until the very moment they occur.

To identify anomalies, it is necessary to recognise behavioural patterns that have never been witnessed before or are clearly distant from what is considered the norm, and it calls for a definition of anomaly detection as a process deeply intertwined with the perception of change.

The first step to take towards the path of anomaly detection is defining what normalcy is, which can be viewed as the degree of pattern consistency throughout the observed states of a system.

In the scope of dynamic systems, anomalous performance induces instability, emerging as a rupture of the order while bringing the system out of its normal state into an abnormal one.

An anomaly detection system is profoundly dependent on the domain it is applied to, so much so that a crucial part of modelling such system focuses on the specific environment in which the anomalies are to be studied. In this perspective, a critical distinction to be made is that of abnormalities from the very background noise or "clutter" of the system, made with the aid of two criteria: selectivity and sensitivity.

These two are respectively used to study the situation from both a macroscopic and microscopic point of view. Selectivity uses a coarse-granularity approach to identify and locate potential areas of interest, allowing the system to shift to the use of sensitivity, which, due to its fine-granularity, is capable of analysing the highlighted area in detail.

A combination of these resources provides the means to shape models of normal background activity, in contrast with anomalous activity.

However necessary automating the process of anomaly detection is, a decisional cycle that includes human operators is equally important, most of all in safety-critical domains. Human supervision is fundamental to avert machines' faults, by intervening and correcting their results in order to represent the real human perception of the event.

A system shaped like this makes use of innate human capabilities of logical

reasoning, memory and adaptability, all essential for the goal of reliably distinguishing between normalcy and anomaly, while also introducing a self-learning path, where the system walks towards enhanced accuracy.

An effective detection operation includes the act of tracking the dynamic system's behaviour that is being studied, which gives useful information to develop the ability to reproduce object trajectory in any frame sequence, making the system able to detect anomalies in real-time and to retrace the scenario to conduct accuracy checks with regard to the normalcy recognition of the system.

1.2.1 Thermal videos

"Infrared thermography (IRT) is a technique that detects the intensity of radiation in the infrared part of the electromagnetic spectrum and visualizes the recorded temperatures as 2D images." [16] The difference in temperature is appreciated by the colours in the greyscale image in the following way, if using the "white hot" palette: from white, representing elevated values, to black for lower values.

There are many advantages to this technology, such as high-precision and providing non-contact temperature measurements. Among the many applications it is suitable for, there is railway infrastructure monitoring, the particular domain taken into consideration during the conduction of this project.

In the context of thermal videos, an anomaly is typically identified as an "hot-spot", whose value exceeds some threshold, recognised by calculating the temperature difference, called "DELTA-T" between a Region of Interest (ROI), which locate the anomaly, and a reference, chosen to be specific to the particular application, or the environment.

Compared to optical images (RGB), thermal images present various challenges for computer vision tasks: lower resolution and signal-to-noise ratio, blurred details, low contrast and lack of texture are all contributing to the need of alternate solutions with respect to a completely automated anomaly detection system, even more so if based on segmentation algorithms trained on coloured datasets.

All this calls for the use of advanced AI models, which have to be trained using robust and tailored ground-truth dataset, in order to address the problem effectively.

1.3 Annotation

1.3.1 Image Annotation

"Image annotation (or image data labelling) is the process of labelling images and adding tags to create datasets for training computer vision models." [7]

As the annotating process consists of highlighting the target and adding context via text-based notes, an annotated image presents one or both of these elements: textual tags, assigning the identified object to a class and eventually adding more specific information; some kind of shape, from the simplest rectangle (bounding box) to a polygon to the most accurate mask, tracing the object, which gives information about its location.

Seeing as it is the pre-training phase for computer vision models, it differentiates into three forms, each specifically aimed at a CV technique: classification, which consists of applying a label or multiple labels to an image as to state the presence of the described item in the label; detection adds a localisation element, indeed, this piece of information is given through the use of the shape element described above; lastly, segmentation increases the accuracy of the spatial knowledge given by annotating, reaching pixel-level granularity and, as in CV, instance segmentation distinguishes between different instances belonging to the same class, while semantic segmentation does not have this ability.

Annotation can be specifically used for supervised learning, a technique where the provided dataset on which the model should learn is composed of input-output pairs that, in this domain, are structured as image-annotated image.

Another application of this process is performance evaluation to assess the accuracy of the model's learning process, for it provides ground truth labels for the computer vision model that can be compared to the output produced by the model.

1.3.2 Video Annotation

Video annotation includes image annotation, as annotating a video frame is the same as annotating an image, but that is just a minimal part of a much more complex task. This process takes raw video footage and, through the act of labelling and identifying specific objects of interest, transforms it into structured data, creating the ground truth from which computer vision models can learn.

Other than location and class, an annotated video must account for an ulterior dimension: time. The annotation must be able to follow the targets around as the video progresses. This piece of information has different meanings and methods of being acquired depending on the nature of the video, the domain one is working in, and the specific application for which annotation is being used. In general, when the target appears or how long the target is present in the video are both relevant pieces of information for the sake of creating a dataset.

In the context of pure annotation, different methods can be used, and consequently, different timestamps are more or less important.

The goal of a video annotation process is that of maintaining an object's identity and spatial accuracy as it moves in time, both achieved by creating an object track. Following the CVAT guide to video annotation[8], there are four main approaches one could take to execute this.

Object tracking has the main objective of identifying and monitoring targets, by incorporating some level of automation into the process. It combines human expertise and machine precision by following this collaborative cycle: first, the human annotator assigns a unique identifier to targeted objects, which will remain persistent through a continuous sequence of frames due to the work of the AI model, which uses pixel-level locking to trace masks and track the previously identified objects. The final step sees the human counterpart as a supervisor, whose job comprises of providing corrective key-frames implementing a proper HITL verification stage.

Interpolation and occlusion management is a technique to handle high volume data or data consisting of complex videos. This workflow, as the one described above presents an automated stage aimed at simplifying the process. The initial phase is used to mark as key-frames those where an object appearance begins, ends or changes path. These are crucial for key-frames interpolation which also accounts for the targets being partially or completely obscured for a time. During this time the track remains active, so that when the object re-enters from behind a visual obstacle it is recognised through the same identifier. Not only this approach ensures continuity but it concentrate human labour on fewer, more significant frames.

Action and event annotation is also know as "temporal segmentation", for it labels behaviour rather than instances, occurring within a time-frame defined between a start and end trigger, marked by the human annotator to be then tracked by an AI model. In addition to the possibility for a single object to be associated with multiple action labels both sequential and overlapping, the video is also analysed as a whole to implement global scene classification, for events happening on a larger scale, or in the background.

Video annotation primitives are used to define spatial boundaries of targets. They are exactly the shapes used in image annotation, the difference being their consistency with the position of moving objects, implemented through the methods mentioned above.

1.3.3 Costs and Challenges

Although data annotation is a time-consuming and costly process, especially if the data consists of high-volume, complex video footage, it is necessary in order to produce curated training sets for computer vision models.

Annotation per se is not sufficient, the dataset needs to be vast, diverse, and tailored to the specific application to avoid a narrow biased perspective while also allowing the model to gain accuracy. The computer vision model must be able to navigate in real-world, often real-time scenarios, and reliably produce accurate results outside of controlled environments, such as the training set.

Quality training is reflected in high-quality annotated datasets, just as feeding a model low-quality data will produce inaccurate results. Quality annotation leads to improved model accuracy, enhanced data consistency, reduced time and costs, minimized biases, and overall more efficient model training.

A common solution for annotation's slow pace, is integrating some level of automation into the process. By using pre-trained AI models, annotation time can decrease a considerable amount, and, if used strategically, it can actually improve the annotated data quality. Contrary to what one could naively believe, completely automating the process is actually counterproductive: removing essential human oversight does not address the need to manage edge cases, ambiguous scenarios, a model's weak spots or blind spots.

However, as convenient as automated annotation techniques can seem, AI models may lack the deep contextual understanding and domain expertise re-

quired, especially if one means to train a model for a specific domain in which computer vision models have not yet been trained enough. Hence the need for a new annotated dataset. This is most true for anomaly detection, especially in safety-critical environments, such as railway infrastructures. Human supervision cannot be sacrificed, establishing the foundation for a symbiotic approach known as the Human-In-The-Loop (HITL) paradigm.

By using different methods, it is possible to mitigate the workload and conveniently concentrating human energy on the very particular tasks it is most needed, by letting machines do the heavy work.

The need for this kind of approach is even more called for, in the specific context of thermal imaging, where relying solely on human operators for manual annotation is not sufficient, just as machines alone cannot reach the desired reliability due to the nature of this data.

1.4 Human-In-The-Loop Paradigm

As mentioned above, human supervision cannot be sacrificed when working in safety-critical domains.

The interaction between humans and machine learning algorithms is studied to find ways to render the process of improving these algorithms more efficient.

Although there are other scopes to be explored where humans can be part of the algorithms' work, for the sake of this project, we will discuss of Human-In-The-Loop Machine Learning (HITL-ML) as a blanket term that covers many different approaches of this technique, which we will be differentiating based on one main characteristic: who is in control. [18]

We talk about Active Learning (AL) when the model holds the reins, Interactive Machine Learning (IML) when the responsibility is shared, and Machine Teaching (MT) when humans are in control.

1.4.1 Active Learning

Active Learning (AL) is a machine learning approach where the learner is in control, as it monopolises the choice of the data that is to be handed over to the human, who is treated as an "oracle" to annotate unlabeled data. It is considered to be a data-driven, semi-supervised learning technique, particularly

useful when one has a vast unlabeled dataset, since manual annotation, however ideal in terms of accuracy and context understanding, is costly.

It comprises of two phases, parts of an iterative process. At first the model selects a number of examples to submit to the oracle: this is the core of sampling, which can be implemented in different ways, based on how many examples are chosen, how many are submitted at a time, and how they are chosen. One particular method worth mentioning, for its relevance to this project, is Pool-based sampling, where the entire dataset is evaluated by the model in order to elect the elements that are most suited for manual annotation. Once the latter is complete and labels are ready, the second phase begins: training. The model gets re-trained on the data that was just annotated, either in batch or sequential mode, depending on whether several instances are labelled before updating the model or each new label triggers a model update.

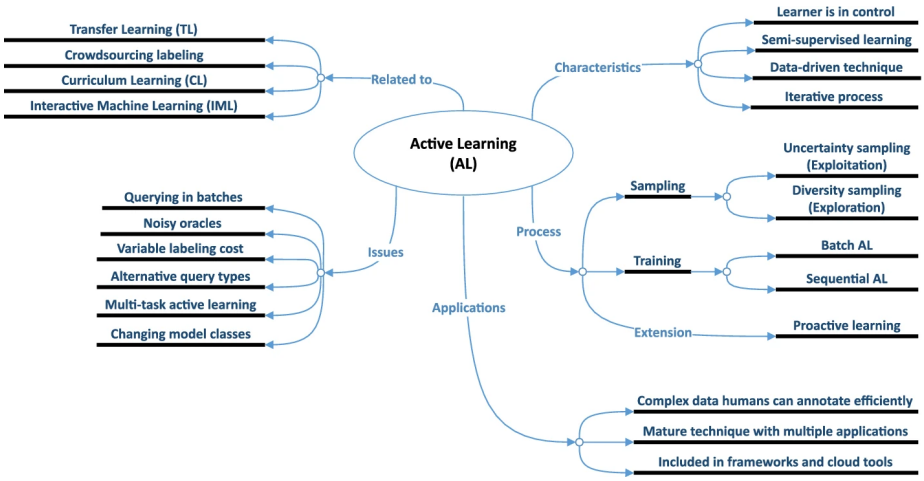

Figure 1: Active Learning conceptual map [18]

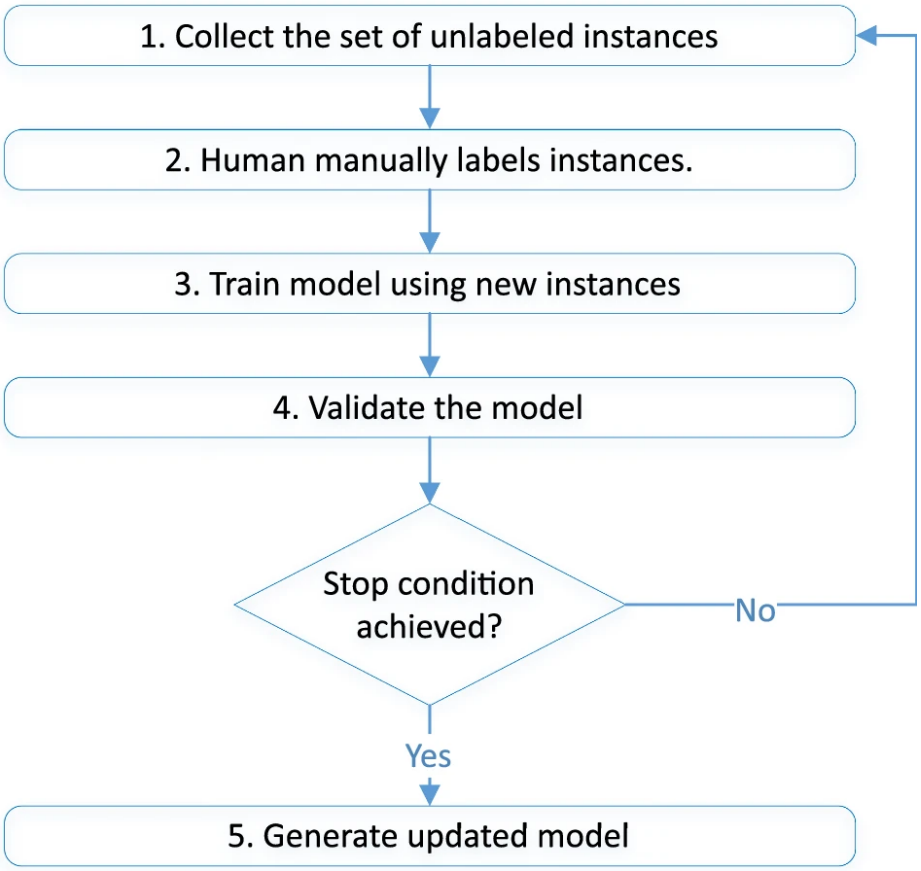

Figure 2: Active Learning process [18]

1.4.2 Interactive Machine Learning

Interactive Machine Learning (IML) includes humans in the ML loop, by sharing the control over the learning process between the user and the system. Here, Human-Computer interaction (HCI) plays a fundamental role: human intervention incrementally supplies information with focus and frequency, in different parts of the process. This bidirectional relationship between the two is continuous, and guarantees valuable feedback exchanges. In fact, the flexibility of the human role is a key characteristic of particular this approach in machine learning. Humans can act at the beginning of the workflow for the identification job, hard for CV algorithms but easy for human brains, or at the end, to validate

or correct the model's produced results.

It can be best applied on unstructured and complex data, such as images and videos.

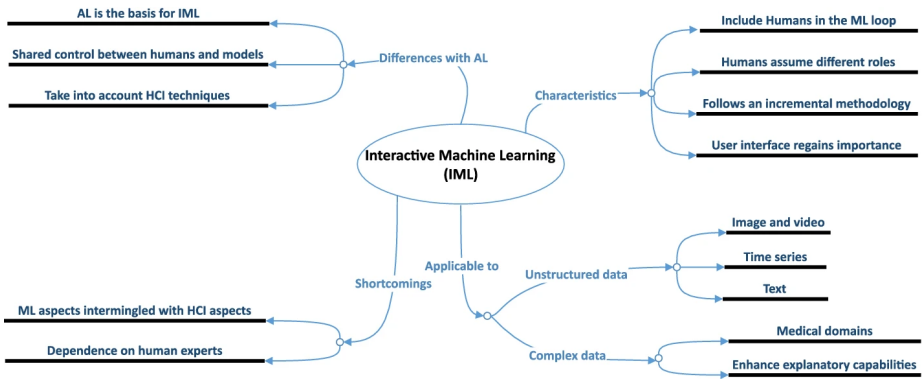

Figure 3: Interactive Machine Learning conceptual map [18]

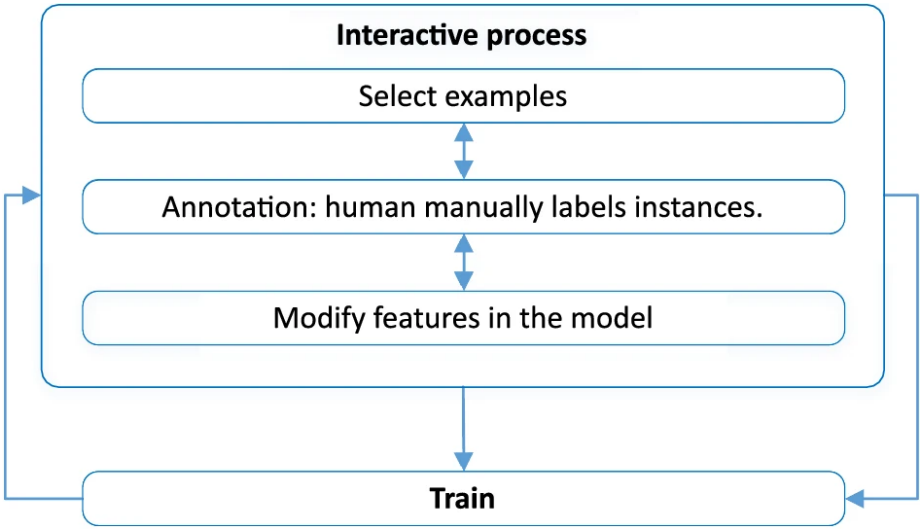

Figure 4: Interactive Machine Learning process [18]

1.4.3 Machine Teaching

Machine Teaching (MT) turns the tide in favour of the teacher. Here, the human teacher has complete control over the learning process, carrying the responsibility of transferring knowledge over to the machine.

The focus falls on the teacher's efficiency, contrary to classic machine learning, where to increase the learner's accuracy the method concentrates on the learner itself. This method is structured so that teachers can be domain experts, with regard to whichever domain the model is being trained in, without the possess of technical machine learning ability.

Here too the information can be fed to the learner in two modalities, batch or sequential. The first rendering the order in which the model sorts through the data unimportant, while the latter presuming the data to be in a structured and optimised way, so that each learnt example can be useful for the next.

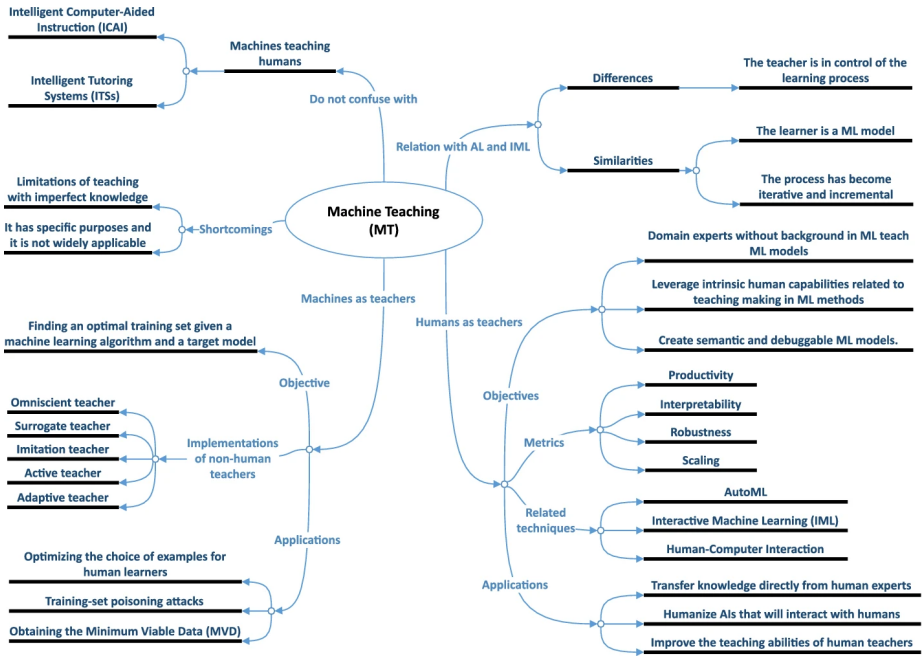

Figure 5: Machine Teaching conceptual map [18]

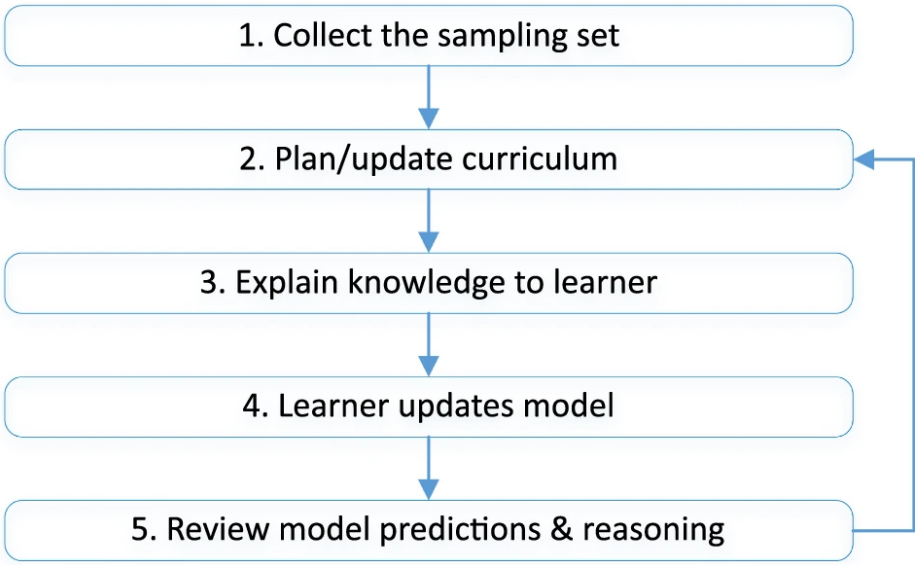

Figure 6: Machine Teaching process [18]

2 Development of the Annotation System

This section aims to explain how the annotation system works, from the idea, through its development, to structure and usage.

2.1 Addressing Annotation Challenges

This system is designed to drastically reduce the human labour involved, focusing its energy in key tasks that are strategically aided by computer vision models.

The user's role is mostly of corrective and signalling nature, in order to give the basis to the chosen models for the actual annotation, intended as the act of drawing masks and bounding boxes. Seeing as the latter would be too simplistic, although they don't require as much work as drawing frame-by-frame polygons by hand, which would be far better suited for the problem, utilising State-of-the-Art segmentation models makes it possible to sacrifice human manual labour in favour of faster machines working at high standards. Another advantage of taking this approach is being able to address video annotation complexity, caused by the necessity of a tracking system, which is conveniently provided by the model, allowing the user to concentrate on revising key-frames.

By specialising in thermal videos, the system validates the necessity to pay attention to the problem of lacking training sets about specific domains. Adding to that, is the ulterior focus on railway infrastructures anomaly detection, which can be defined as safety-critical.

The computer vision industry is transitioning away from traditional pure manual labelling toward AI-assisted workflow, changing the role of the human operator from drawer to reviewer. Compatibly with this tendency, the project presented utilises HITL methods in favour of an efficient curation of high-quality, domain-specific datasets.

2.2 The HITL Approach in the System

The annotation system developed for this thesis implements, to varying degrees, all three Human-In-The-Loop techniques described in section 1.4.

An Active Learning strategy is utilised during the preliminary scanning of the video. The RFI anomaly detection algorithm identifies `suspect_frames` and presents them to the user to oversee. This directs the user to informative data to simplify the beginning of the annotation process.

There is also a heavy reliance on the Interactive Machine Learning technique. The core models (SAM 3 and YOLOE-26L-Seg) depend on direct user action, consisting of planting positive and negative seeds on the image or providing textual prompts, to generate and refine their predictions dynamically.

The ultimate objective of this application is to create an accurately annotated dataset ready to be used for model training, making human presence in the loop necessary. This structural need for human oversight and correction embodies a Machine Teaching paradigm: human knowledge of the anomaly domain can thus be systematically transferred to the model, guiding it toward the desired ground truth.

2.3 System Architecture

The system architecture takes into consideration two fundamental requirements: handling the heavy computational load needed by modern artificial intelligence models while keeping high reactivity and fluidity to ensure a smooth user experience.

The system includes a hardware infrastructure and a modular software architecture.

2.3.1 Hardware Infrastructure

Figure 7 shows a general overview of an adequate hardware for the application to run on.

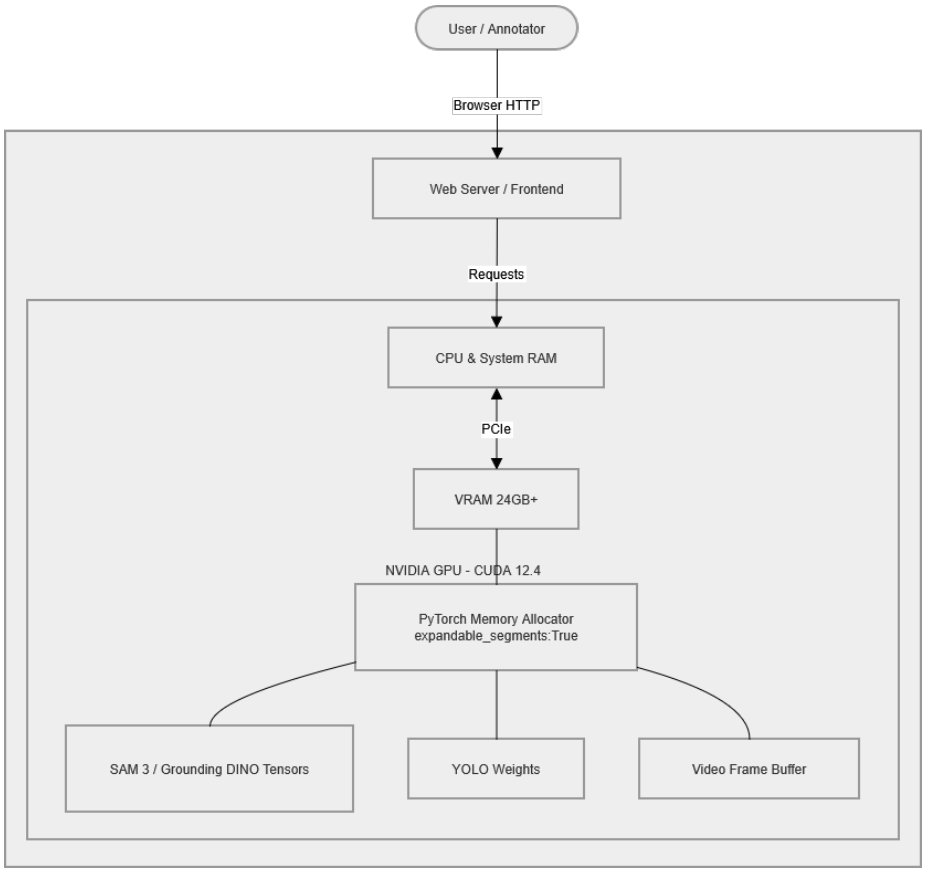

Figure 7: Hardware Architecture Block Diagram

Real-time video processing demands good performance levels on its own, more so if execute by advanced computer vision models, as SAM 3 and YOLO26 are.

To ensure that the hardware infrastructure is performant to the necessary extent for launching the models used in this application, we must account for two main components: Graphics Processing Unit (GPU) and Video Random Access Memory (VRAM).

GPU

During the development and testing of the application, multiple GPUs have been used, the weakest being Colab's T4 [12], which was utilized in the initial

stages of the development when the application was structured as a Jupyter notebook. After transitioning to the RunPod [25] platform, three GPUs were sequentially employed. RTX 2000 Ada, RTX 4090 and ultimately, the system was run on RTX PRO 4500.

The system is thus dependent on a NVIDIA GPU and is optimised for modern architectures as RTX 4000 or Blackwell, through CUDA 12.4 drivers.

VRAM

Video memory management is most critical for this system due to the simultaneous instantiation of different models weights, joined with frame by frame object tracking, causing rapid memory saturation.

To mitigate VRAM fragmentation and to prevent Out-Of-Memory (OOM) errors or process deadlocks, the infrastructure exploits specific environment variables on an Operating System level, for instance `PYTORCH_CUDA_ALLOC_CONF =expandable_segments:True`, making it possible to optimise dynamic memory allocation using PyTorch.

2.3.2 Software Architecture

Figure 8 shows a general overview of the software's structure.

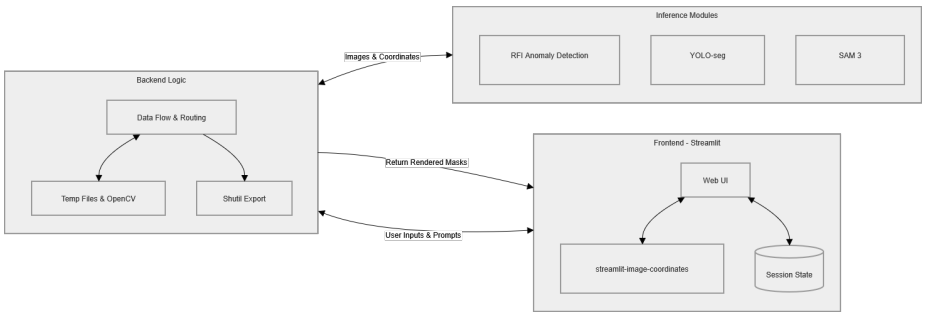

Figure 8: Software Architecture Block Diagram

The software has been developed using the Python programming language, following a modular approach to draw a clear line between user interface and the system logic.

To best guarantee portability, the system deployment is supported via both isolated Docker [9] containers and Conda [2] virtual environments.

Figure 9 gives an introductory definition of software requirements and of the Human-Computer interaction which lays the basis of the functioning of the application, by illustrating main functionalities and actors involved.

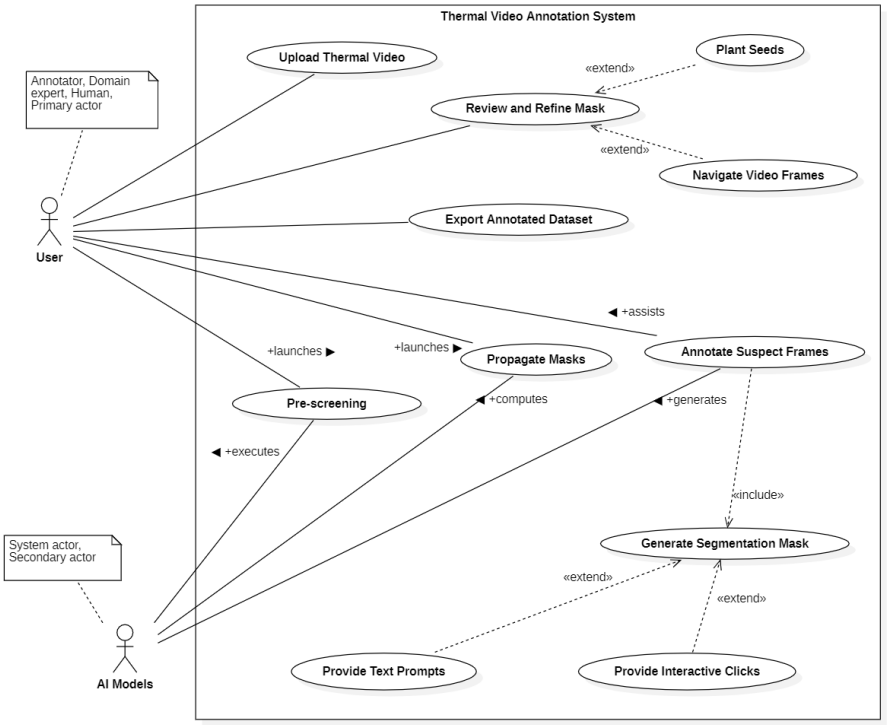

Figure 9: Use Case Diagram

Frontend (Interface)

The Web User Interface (Web UI) was developed using the Streamlit framework [26]. Integration of the streamlit-image-coordinates library to enable the HITL (*Human-In-The-Loop*) paradigm allows the system to capture the coordinates of the users' clicks.

Backend (Logic and Data Flow)

Data flow of the application is managed sequentially, starting when the video gets uploaded, and two actions are executed: the storing of the video into a temporary file, using the tempfile library [24], and the extraction of the first frame, referred to in the application as `frame 0`, which is handled by the OpenCV library [5], as all that concerns video management.

Other than organising data, the backend manages what happens behind the two most important buttons on the app: the "pre-processing" button and the "processing" button, that respectively hide the core modules of RFI Anomaly Detection [14] which executes the pre-screening process, while the other takes care of inference and tracking by using two models, YOLOE-26L-Seg [28] or SAM 3 [3], in order to generate masks.

Once the masks have been generated and drawn onto the frames, they are also converted to the standard YOLO format (.txt) and aggregated into a compressed folder with the use of the shutil library [21].

Finally the video is recompiled to visibly apply the annotation onto the frames shown via the interface for the final step of the loop, revision and correction.

Memory Management

Web applications need to implement mechanisms dedicated to maintain their state, let losing important information a every page rerun due to the stateless nature of the HTTP protocol, which is stateless by nature.

This responsibility falls on Streamlit's `session_state`, which enables a temporary memory mechanism to save and maintain crucial variables `session_state`: to maintain variables when the page is rerun, through the employment of helper methods `init_state(key, default_value)` and `reset_state()` to, respectively, initialise the variables to default values and set them to the same default values again when needed.

- `positive_seeds_x` - dictionary maintaining positive seeds associated to each annotated frame, one for the original video (`og`) and one for the processed video (`ann`).

- `negative_seeds_x` - dictionary maintaining negative seeds associated to each annotated frame, one for the original video (`og`) and one for the processed video (`ann`).
- `last_click_x` - keeping memory of selected seed type, for original and annotated video.
- `x_video_path`
- `selected_video` - to keep track of which video is being annotated, processed and navigated.
- `total_frames`
- `suspect_frames` - frames presented by the pre-screening process.
- `cf_x` - current frame, relative to the selected video.
- `sync_nav` - to toggle between synchronised and isolated navigation.
- `loop_iteration` - to keep track of the iteration of the interactive annotation loop.
- `output_video`
- `output_zip`

Persistent Label Accumulation

To effectively store annotated data through the process and avoid the risk of overwriting annotations at each iteration of the processing module, a backend logic defined as Persistent Label Accumulation was engineered.

This mechanism acts as a "master accumulator", providing the system with historical memory across multiple Human-In-The-Loop iterations, by operating on three core principles.

Progressive annotation is the mechanism enabling the build up of a multi-class ground truth datasets frame by frame: at each iteration the system access the existing .txt file of newly annotated frames to append new labels, dynamically merging labels across the inference loop.

Duplicate prevention is ensured by the use of `list(set())` during the accumulation phase which discards any redundant line before writing to the master database.

Comprehensive historical export, implemented when triggering the "Download annotations" button, consists in bypassing the temporary folders of the latest iteration and packaging the entire master database to deliver a complete, refined dataset with annotations related to anomalies detected during every phase of the interactive session.

2.4 User Workflow

As is visible in figure 10, user workflow is structured in sequential phases that ultimately constitute a HCI loop between the operator and the integrated models.

Here is a high level overview of the workflow from the user's point of view, before going into detail and describing each phase.

The process begins with the uploading of the video file, followed by the choice between launching the pre-screening module, in which case the user can navigate through the suggested suspect frames, or not. The user must then choose which model to use for segmentation, after having provided the appropriate inputs, which can take the form of text or clicks ("seeds") deposited directly onto the frame via interface. Once the model has been launched, the user waits until the annotated video appears onto the screen: if the user deems to be satisfied by the results, the only thing left to do is downloading the annotations using the dedicated button, otherwise the user continues from manual annotation and choosing the model.

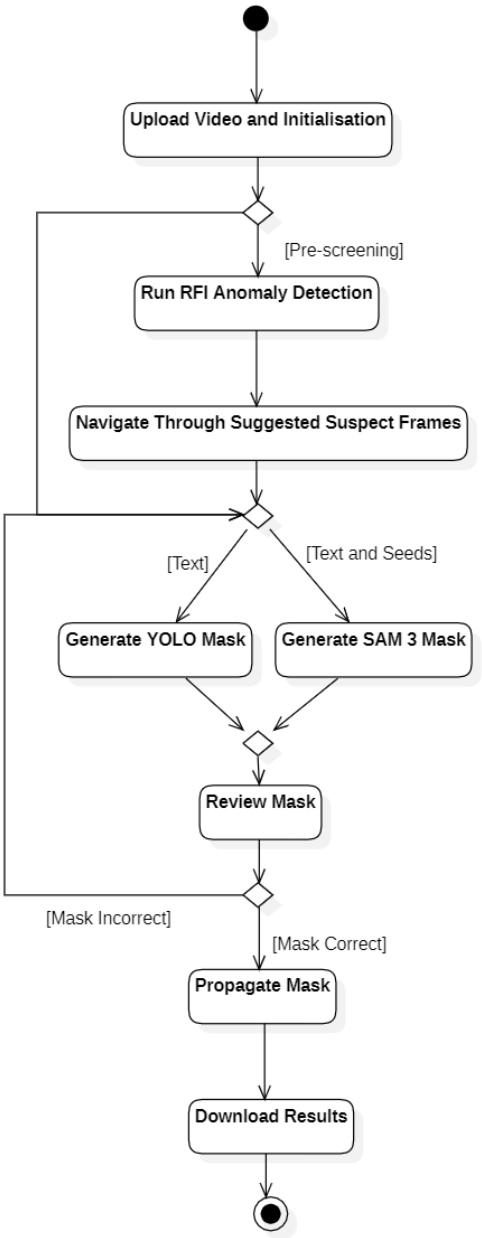

Figure 10: User Workflow

2.4.1 Phase 1: Video Upload and Visualisation

The first step for the user to take is uploading a raw thermal video using the designated button situated in the sidebar `MENU`.

Once the file is uploaded, the system immediately responds by extracting the first frame, shich will appear on sceen.

2.4.2 Phase 2: Automated Pre-screening

At this point the user can choose to activate the pre-screening, automated by using the RFI Anomaly Detection model.

Once the user has pressed the "Start RFI Anomaly Detection" button, the Active Learning paradigm activates: the system temporally takes control to select significant frames and store them into a list to be presented to its oracle in the form a drop-down menu.

The user can now navigate the video from one suspect frame to another, by simply scrolling through the options.

2.4.3 Phase 3: Interactive Annotation

Guided by the system suggestions, in case the user has previously chosen to use the pre-processing module, the manual annotation process begins.

The user can navigate the video in its entirety by utilising the slider to jump to any frame, or via the drop-down options of suspect anomalous frames.

Interactive Machine Learning really takes place once the user starts annotating, which can be done in two modalities: prompt-based and coordinate-based.

The Prompt-based Path This is the path chosen to use the YOLO model by exploiting its open-vocabulary architecture, or the SAM 3 model by leveraging its Promptable Concept Segmentation approach.

After selecting the model, the user inserts as many custom classes as seems fit and locks them in by pressing enter. They will be the basis upon which the model will generate masks and bounding boxes.

The coordinate-base Path This is the path chosen to use the SAM 3 model, taking advantage of the highly precise and interactive nature of the inputs given to the algorithm in the form of coordinates.

After selecting the model, the user clicks directly onto the chosen frame to "plant seeds" from which the model will grow masks, in case of a positive seed, or pierce through them if the seed is negative.

The positive seeds appear green, while the negative ones are red, to make it clear at first glance what kind of action the user is performing; they can be chosen via a Streamlit radio tool presenting the two options.

2.4.4 Phase 4: Automated Annotation

The user launches the chosen model after having given the proper inputs.

Once the models have completed the processing of the video, the results are shown on the interface with the same mode the original video is, exactly under it. The annotation are also available for download, as annotated video and zip folder.

2.4.5 Phase 5: The Loop

The user can now dedicate oneself to the review and refinement of the generated mask, using the seeds just as before, to then launch SAM 3 which will take the corrections into consideration when re-analysing the video, implementing the Machine Teaching technique.

This process can be repeated as many times as the user thinks necessary to reach a satisfying result, with respect to the requirements imposed by domain, application, and the many aspects discussed in previous sections.

2.4.6 Phase 6: Output Export

The user can export the output in the form of an annotated video in .mp4 format and zip folder containing the .txt annotations.

2.5 Models Integration

The heart of the system dwells in the integration of three artificial intelligence models, each specialised for a specific phase of the complex annotation process.

2.5.1 Integration of RFI Anomaly Detection

This model serves as pre-processing module, applying detection heuristics and logic to identify thermal anomalies in the specific context of railway infrastructure.

Launching

RFI Anomaly Detection gets executed as a separated process by utilising the subprocess Python library [23], which isolated execution and catches output in real time.

Listing 1: RFI_anomaly_detection Integration - app.py

```
ad_results = subprocess.Popen(
    ["python", "-u", "anomaly_detection.py", "--input_video",
        st.session_state.og_video_path], #u: unbuffered output
    cwd=os.path.join(BASE_DIR, "RFI_anomaly_detection"),
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    text=True,
    bufsize=1,
    env=env
)
```

Library Management and CUDA

To guarantee the module succeeds in accessing the hardware acceleration, CUDA libraries' paths are dynamically configured in `LD_LIBRARY_PATH` by adding `.so` libraries into the subprocess' environment and by finding nvidia packages with the aid of the site Python library [22].

Modifications

To enable bidirectional communication with the Streamlit interface, some modifications were made to the original code found in its GitHub repository, which can be consulted in the `patch_files` folder of this application's GitHub repository: every alteration has been marked with a `#CHANGE` comment in order to make it easier to find and recognise them.

The patching process has been automatised by the `cp patch_files/...` command found in the setup.sh script.

- **Output Management**: anomaly_detection.py has been altered to explicitly save the indexes of frames where an anomaly first appear in the video, into the found_anomalies.txt file which is then retrieved in app.py by the `extraxt_anomalous_frames()` function.
- **Memory Cleanup**: lest saturating server storage in case of multiple executions, it has been introduced an automated cleanup routine to remove the content of `temp_safe_obstacles_dir` and `temp_dangerous_obstacles_dir` at the end of an execution.
- **Metrics**: `safe_div` to ensure no divisions by 0 are executed during the evaluation logic of Intersection over Union (IoU) and to avoid crashes in situations of detection fuse overlapping.

Listing 2: extract_anomalous_frames function - app.py

```
def extract_anomalous_frames(file_path):
    full_path = os.path.join("RFI_anomaly_detection", file_path)
    if not os.path.exists(full_path):
        st.warning("No anomaly file produced.")
        return []
    with open(full_path, 'r') as file:
        rows = file.readlines()
        frames_list = [int(row.strip()) for row in rows
            if row.strip().isdigit()]
        return frames_list
```

2.5.2 Integration of YOLOE-26L-Seg

YOLOE-26L-Seg provides instance segmentation based on textual prompts, with an open-vocabulary approach: the model converts text into feature vectors, localises the object using the bounding box tool and applies a dedicated segmentation head to extract the mask inside of the box.

The integration of the model leverages the Ultralytics library [27] APIs.

Listing 3: YOLOE-26L-Seg Integration - app.py

```
results = model.predict(
```

```
    source=target_video,
    conf=0.15,
    save=True,
    save_txt=True,
    project=os.path.join(BASE_DIR, "results"),
    name="yolo_out",
    exist_ok=True,
    stream=True
)
```

Additionally, to guarantee reliability during the deployment, the model's cache is dynamically configured through the `os.environ[\YOLO_CONFI_DIR"]` instruction, which redirects YOLO's configuration path into a local folder.

2.5.3 Integration of SAM 3

Sam 3 also provides instance segmentation, firstly using text prompts to assume the same functionality as the YOLO model.

To implement the processing of the video with approach, the execution gets launched through the Ultralytics library.

Listing 4: SAM 3 Integration (text) - app.py

```
from ultralytics.models.sam import SAM3SemanticPredictor
overrides = dict(
    conf=0.25,
    task="segment",
    mode="predict",
    model="sam3.pt",
    save=True,
    save_txt=True,
    project=os.path.join(BASE_DIR, "results"),
    name="sam_out",
    exist_ok=True
)
predictor = SAM3SemanticPredictor(overrides=overrides)
results = predictor(
    source=target_video,
    text=prompt_list,
```

```
    stream=True
)
```

The other kind of input SAM 3 takes is coordinates, and its integration can be done in two different ways, which are described below. The ultimate version of the system uses the stateful approach.

The Stateless/Multi-Pass Approach

Using high level Ultralytics APIs is the more direct way: the models gets launched by calling the `predict()` method of the model, which applies the given coordinates to the first frame of the given video.

As it goes forward into the video processing, SAM 3 has no memory of previous frames, hence the definition of this method as "stateless".

For this very reason, the solution found to make SAM 3 able to track anomalies that appear for the first time in different frames was that of trimming the videos in the following manner. Since the seeds planted by the user also give temporal information, intended as the appearing of the anomaly, in addition to the spatial information, they are used to find the first frame where the anomaly has been manually detected by the user to use as starting frame for the trimmed video that is to be used as input for the model.

This process gets repeated for each anomalous frame, creating several separate masks, which are overlapped only at the end and then applied onto the whole video.

It's quite evident how this approach can easily become very computationally heavy, as the model ends up revisiting the same portion of the video multiple times.

It can be an acceptable solution if the dataset is made up of videos where there are rare anomalies that let a considerably large number of frames pass between each other.

The Stateful/Single-Pass Approach

This approach is based on a low level access into SAM 3's motor: the system directly interacts with the model's `inference_state` gaining the ability to associate the seeds to specific frames.

The model constructs a memory bank to store information about the analysed objects into, as the analysis proceeds. SAM 3 can track any inputted anomaly appearing in any frame at first glance.

To give steady support to this approach and avoid the memory bank saturation, the system globally forces the `PYTORCH_CUDA_ALLOC_CONF` variable to the value `\expandable_segments:True"` variable the in app.py and .env files: this optimises PyTorch dynamic allocation and prevents VRAM fragmentation.

3 Models Presentation

3.1 RFI anomaly detection

The RFI ("Rete Ferroviaria Italiana", Italian Rail Network) Anomaly Detection system aims to achieve zero-shot recognition of anomalies on railway tracks in thermal videos, by leveraging two models in order to rely on a two-stage decoupled pipeline known as Grounded SAM 2. The two models involved are indeed Grounding DINO and SAM 2.

The first acts as the semantic locator by aligning image features with language features, by virtue of the Transformer-based architecture of DINO merged with grounded pre-training. Being an open-set object detector, Grounding DINO takes the inputted text prompts to localise it with no require of predefined classes and outputs bounding boxes around the targets.

For this specific use, the first object it tries to identify is the background and context of the scene, i.e. the railway and ground, referred to as `safe` elements, to then transition into isolating the `dangerous` elements: the anomalies.

SAM 2 constitutes the second phase of the sequence, taking over once the anomalies have been detected, and their location defined. This model receives the bounding boxes generated by Grounding DINO as spatial prompts and utilises its internal memory bank and temporal attention mechanism to generate masks at pixel-level precision and propagate them across succeeding frames.

This chained approach allows a segmenter like SAM 2, lacking of text prompt support, to operate a zero-shot video segmentation process.

3.2 YOLOE-26L-Seg

YOLO26 represents a paradigm shift for the You Only Look Once (YOLO) Family, and was engineered to maximise deployment efficiency in dynamic environments by introducing a native end-to-end NMS-free inference.

Relying on Non-Maximum Suppression (NMS) as a post-processing step to filter out overlapping bounding boxes causes latency and computational overhead: eliminating it in favour of directly outputting the final predictions significantly improves the situation.

YOLO26 also addresses the discrepancy between theoretical computational complexity and actual inference speed on edge hardware, called "Export Gap", by completely removing the Distribution Focal Loss (DFL) module thus simplifying the architecture and speeding CPU inference by 43% [11].

Innovation was also brought in training: the introduction of a hybrid optimiser inspired by Large Language Models (LLM) enhanced training stability, and with the YOLOE extension and the introduction of the open-vocabulary paradigm, YOLO attempts to overcome the limitations imposed by using a fixed classifier head.

Compared to the original closed-set approach, limiting the detection to a fixed number of predefined classes the model has been explicitly trained on, zero-shot text prompting allows the model to dynamically map linguistic concepts to visual features from the provided textual descriptions.

To achieve instance segmentation, YOLO architectures extend the standard object detection pipeline to integrate a dedicated mask branch which works in parallel with the bounding box predictor that delimitate the generated prototype masks, after they have been combined with per-object mask coefficients.

The specific variant integrated into the annotation system is YOLOE-26L-Seg [4], where "L" refers to the neural network's (NN) Large-scale configuration, working to keep an acceptable trade-off balance between high-fidelity feature extraction and real-time inference speed.

By combining the base YOLO26 NMS-free architecture, the Open-Vocabulary extension, and the dedicated segmentation head, YOLOE-26L-Seg achieves zero-shot instance segmentation.

3.3 SAM 3

Meta's Segment Anything 3 represents in fact the state-of-the-art for foundation models in visual segmentation.

Compared to its predecessors, solely relying on spatial prompts which give no information about the target's identity, SAM 3 gains to ability to understand what it is that is being segmented, by creating an association to linguistic concepts.

This model introduces a new approach to segmentation to add to the Promptable Visual Segmentation (PVS), already existing in SAM 2: Promptable Concept Segmentation (PCS).

SAM 3 relies on a unified framework supporting open-vocabulary text prompts, visual prompts such as point or bounding boxes, or a combination of the two in input, to present uniquely identified segmentation masks associated to each instance of the detected objects.

Building upon SAM 2 ability to segment single objects per prompt, SAM 3 is capable of detecting and tracking all instances of the concept given in input, also improving the interactive nature of visual segmentation by providing interactive exemplar refinement of the generated masks making dataset annotation a suitable application for this model.

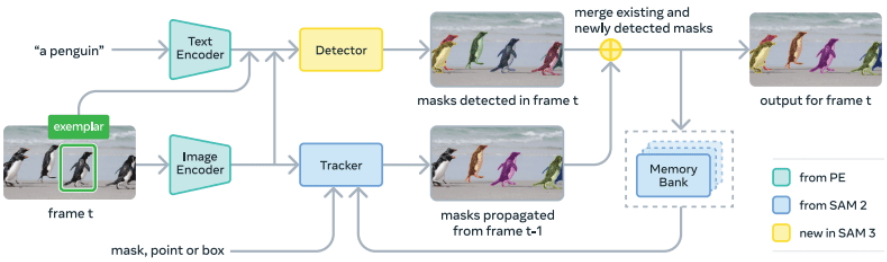

Figure 11: SAM 3 architecture overview [15]

SAM 3 architecture is based on a detector and tracker, whose shared backbone is the Perception Encoder. The latter's dual encoder design provides the necessary means to process text and image exemplar inputs both separately and together due to the presence of the fusion encoder.

The other fundamental element to consider it the presence head which bring an ulterior innovation: decoupling recognition from localisation by using a learned global predicting whether the target concept is present or not in the scene.

Figure 11 highlights the technologies that SAM 3 borrows from SAM 2 compared to the ones introduced in the new model. Indeed, SAM 3 inherits SAM 2's memory-based video segmentation and tracking which includes prompt, mask and memory encoders, a memory bank to store the objects' characteristics across frames, and applies temporal disambiguation.

This model presents some limitations, tied in particular to PCS, like facing phrase complexity or handling ambiguity of concepts or a limited vocabulary scope with regards to rare concepts. Lastly, its computational requirements are pretty demanding, seeing as it is a complex model.

4 Conclusion

The development of this semi-automatic annotation system begins to address the core challenges outlined in the initial sections of this paper: the lack of visual features in infrared thermal imagery, the prohibitive costs of manual video annotation and the non reliability of fully automated systems.

The implementation of a Human-In-The-Loop architecture establishes an effective synergy between human cognitive adaptability, which is crucial for defining normalcy and anomalies in a safety-critical domain as railway networks, and the computational speed of modern state-of-the-art visual foundation models.

The fundamental take, however, is about how this application can be further developed and improved.

4.1 System Considerations

The application managed to bridge the gap between heavy hardware requirements, caused by the integration of multiple computer vision models, considering the size of SAM 3 or Grounded SAM 2, and a fluid and persistent user experience.

The employment of optimised memory allocation via Pytorch serves this very purpose, by mitigating VRAM fragmentation and preventing deadlocks during continuous tracking operations. As does the employment of the subprocess Python library lest weighing down the Streamlit web application when launching RFI anomaly detection.

Furthermore, the intensive use of Streamlit's session state allowed the system to bypass the inherently stateless nature of web applications, ensuring information persistence regarding coordinate arrays, iterative masks, and frame navigation.

The structure of the web user interface follows the workflow in a top-down stream, starting from the pre-screening phase to the manual and at last automated annotation, all this while keeping the focus on the image in exam that is kept centred at all times.

4.2 Models Analysis

4.2.1 RFI Anomaly Detection

The decoupled nature of Grounded SAM 2 offers a layer of granular control that monolithic models currently lack. In Grounding DINO, the user can independently adjust the `text_threshold` and the `box_threshold`, which resulted useful during RFI integration.

Its specialisation in railways infrastructure proved to be useful, as theorised in the first section, in that specific domain. On the other hand, its usefulness is restricted from a wider standpoint.

One thing to note is that RFI Anomaly Detection is a segmentation model per se, but has not been integrated for this purpose in the system.

4.2.2 The Segmentation Models

Throughout the development clear distinctions emerged regarding the roles, capabilities, and limitations of the two primary segmentation models when deployed in the thermal domain, dictating that the direction of an eventual further development of this application may take.

In both cases, the main challenge originates from the zero-shot nature of the open-vocabulary text prompt approach: associating concepts to objects existing into the thermal domain is not an easy task to achieve for models trained on optic RGB imagery.

YOLOE-26L-Seg

Operating on a zero-shot text prompting basis, this model proves to be adequate for fast localisation and coarse-grained results.

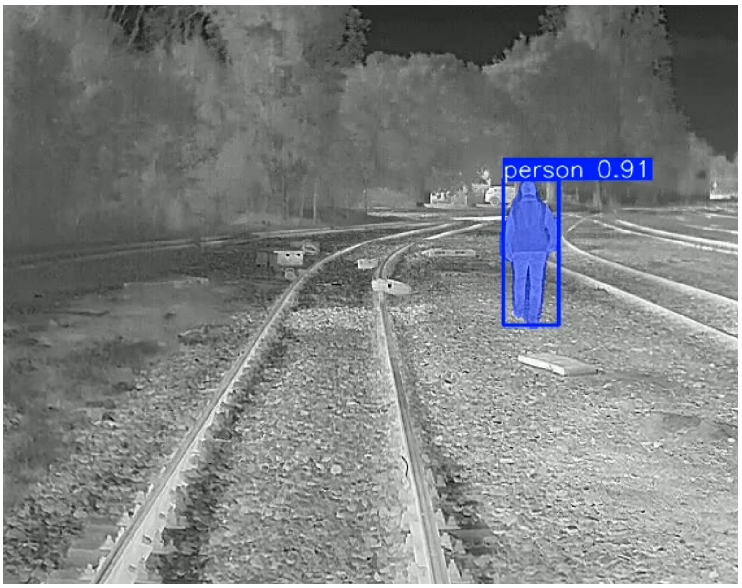

Figure 12: YOLOE-26L-Seg annotated test image based on text prompting

It is indeed effective as an automated first pass to isolate regions of interest in general, although its efficiency is limited by the blurred borders typical of the thermal domain, rendering identification of object not so easily distinguished of little use.

A brief documentation of failed attempts at recognising toolboxes, referring to figure 13.

- toolbox,
- rectangle,
- bright object,
- hot object,
- hot rectangular box.

Figure 13: Test image - San Donato railways

SAM 3

This segmenter provides finer sensitivity with respect to the model discussed previously: the pixel-perfect mask extraction and temporal object tracking make it an optimal option for the annotation applicable.

SAM 3 evaluation is based upon its two prompting modes implemented in the system.

The use of promptable concept segmentation for an initial scanning of the scene in the in an effort to identify anomalies, both locally and semantically, yielded the same limitations inherent to the thermal domain.

The refinement and tracking phase is where SAM 3 demonstrated its real capabilities, even though in particularly blurred and low contrast regions even this model struggled to create an accurate mask for the indicated object, as is evident in figure 14.

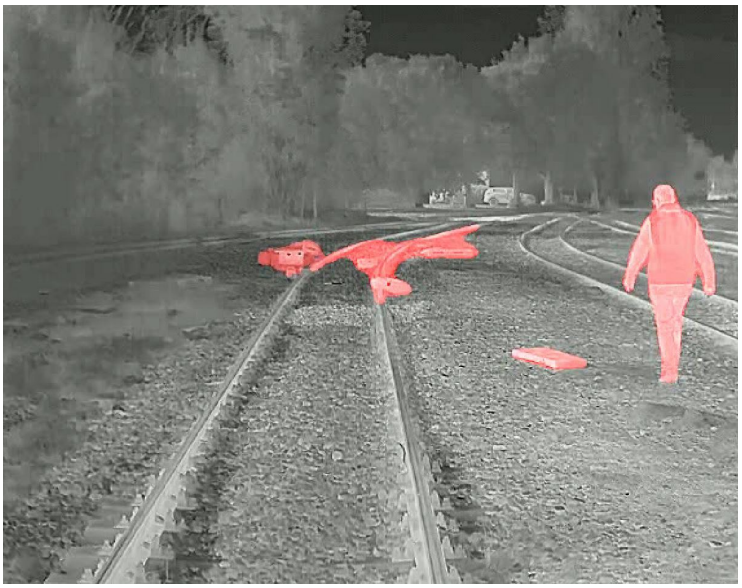

Figure 14: SAM 3 annotated test image based on seeds prompting

The interactive nature of this mode of operating is exactly what a semi-automated annotation workflow like the one developed for this thesis needs.

It allows a smooth HCI, both from a purely functional point of view and from a user experience analysis of the application: the user can immediately see the effects of applied corrections.

Ultimately, rather than viewing these models as mutually exclusive competitors, the system proves that they serve complementary roles: YOLO provides rapid geometric bounding based on text, while SAM 3's stateful coordinate tracking delivers the pixel-perfect precision required for a Ground Truth dataset.

IOU Evaluation

creazione di minidataset ground truth evaulation_iou.py confronto risultati con metrica iou rispetto all gt

4.3 Future Expansions and Improvements

Based on observations collected during the development and testing phases, technical and functional possible expansions are proposed to augment the sys-

tem's power and improving the user experience.

Instance Identification

SAM 3 capability to maintain unique instance identity persistent can be explored to test the possibilities of bringing it to a user level, by tying this identity to the seed planted by the user, enabling the human operator to intervene in tracking, disambiguating between instances of the same concept, and handling objects being obstructed or going out of frame throughout the video.

Data Engineering

Transitioning from local file storage to a database integration, for an improved and better organised management of the dataset creation.

Sequential Pipeline

Emphasising a clear distinction between the first pass and the refinement phase could optimise results and allow the system to leverage precise characteristics of each model in order to create better curated and targeted phases for the application's workflow

Although this intention has already been hinted in the presentation of user workflow and system structure, the actual application does not impose this.

The core idea is to utilise text-based prompt exclusively in the first iteration of the annotation loop, letting the user choose between a faster coarser approach and a more refined, demanding one, and only then is the user allowed to transition into geometrical inputs.

Refining tools Adding a brush and eraser tool to manually refine masks in the final stage of the annotation, to contrast the challenge in producing pixel-perfect masks in the tehrmal domain, where contrasts are low and outlines are blurred.

References

[1] Anaconda, Inc. *Conda Installation — Conda documentation*. https://docs.conda.io/projects/conda/en/latest/user-guide/install/. 2026.

[2] Anaconda, Inc. *Conda Software Distribution*. https://docs.conda.io/. 2026.

[3] Anonymous authors. "SAM 3: Segment Anything with Concepts". In: *Submitted to ICLR 2026*. Paper ID: 4183, under double-blind review. 2025. URL: https://openreview.net/forum?id=r35clVtGzw.

[4] Marco Bertini. *Realtime Detection YOLO26*. URL: https://github.com/mbertini/realtime-detection-yolo26.

[5] Gary Bradski. "The OpenCV Library". In: *Dr. Dobb's Journal of Software Tools* (2000).

[6] Nicolas Carion et al. *SAM 3: Segment Anything with Concepts*. 2026. arXiv: 2511.16719 [cs.CV]. URL: https://arxiv.org/abs/2511.16719.

[7] CVAT Team. *Image Annotation Explained: What It Is & How To Do It Right*. 2026. URL: https://www.cvat.ai/resources/blog/image-annotation-guide#what-is-image-annotation.

[8] CVAT Team. *The Ultimate Guide to Video Annotation for Computer Vision (2026)*. 2026. URL: https://www.cvat.ai/resources/blog/video-annotation-guide.

[9] Docker Inc. *Docker: Accelerated Container Application Development*. 2026. URL: https://www.docker.com/.

[10] Mohamed Elgendy. *Deep learning for vision systems*. Manning, 2020.

[11] eng-mhasan. *Grounded SAM in 2026: Why It Still Matters (Even in the SAM3 Era)?* https://eng-mhasan.medium.com/grounded-sam-in-2026-why-it-still-matters-even-in-the-sam3-era-15315532365a. 2026.

[12] Google. *Google Colaboratory*. 2026. URL: https://colab.research.google.com/.

[13] IDEA-Research. *Grounded-SAM-2*. 2026. URL: https://github.com/IDEA-Research/Grounded-SAM-2.

[14] Alessandro Innocenti. *RFI Anomaly Detection*. Version master. URL: https://github.com/Aleinno001/RFI_anomaly_detection/tree/master.

[15] LearnOpenCV. *SAM 3: What's New?* https://learnopencv.com/sam-3-whats-new/. 2026.

[16] Cai Lile and Li Yiqun. "Anomaly detection in thermal images using deep neural networks". In: *2017 IEEE International Conference on Image Processing (ICIP)*. IEEE. 2017, pp. 2299–2303.

[17] Meta AI. *Segment Anything Model 3 (SAM 3)*. 2026. URL: https://github.com/facebookresearch/sam3.

[18] Eduardo Mosqueira-Rey et al. "Human-in-the-loop machine learning: a state of the art". In: *Artificial Intelligence Review* 56.4 (2023), pp. 3005–3054.

[19] Nadipuram R Prasad, Salvador Almanza-Garcia, and Thomas T Lu. "Anomaly detection". In: *Computers, Materials, & Continua* 14.1 (2009), p. 1.

[20] Daniele Pretolesi et al. "Adaptive Realities: Human-in-the-Loop AI for Trustworthy XR Training in Safety-Critical Domains". In: *Multimodal Technologies and Interaction* 10.1 (2026), p. 11.

[21] Python Software Foundation. *shutil — High-level file operations*. 2026. URL: https://docs.python.org/3/library/shutil.html.

[22] Python Software Foundation. *site — Site-specific configuration hook*. https://docs.python.org/3/library/site.html. 2026.

[23] Python Software Foundation. *subprocess — Subprocess management*. https://docs.python.org/3/library/subprocess.html. 2026.

[24] Python Software Foundation. *tempfile — Generate temporary files and directories*. 2026. URL: https://docs.python.org/3/library/tempfile.html#module-tempfile.

[25] RunPod Inc. *RunPod: Cloud Platform for AI and Machine Learning*. 2026. URL: https://www.runpod.io/.

[26] Streamlit Inc. *Streamlit*. 2026. URL: https://streamlit.io/.

[27] Ultralytics. *Ultralytics Documentation*. 2026. URL: https://docs.ultralytics.com/.

[28] Ultralytics. *YOLO26 Documentation — Ultralytics YOLO Docs*. 2026. URL: https://docs.ultralytics.com/models/yolo26.

[29] Ultralytics Team. *SAM 3: Segment Anything with Concepts - Ultralytics YOLO Docs*. 2026. URL: https://docs.ultralytics.com/models/sam-3/.

[30] Jiaqian Yu et al. "Learning generalized intersection over union for dense pixelwise prediction". In: *International Conference on Machine Learning*. PMLR. 2021, pp. 12198–12207.

[31] Yunhui Zhu and Buliao Huang. "Pixel-level Quality Assessment for Oriented Object Detection". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 40. 16. 2026, pp. 14005–14013.